



## Internet Programciligi II

Ruby on Rails — Hafta 2-10 PDF arsivi, kapsulleme, migration, Devise. 7 gunluk sinav plani ve interaktif quiz.

[Haftalik Yol](#)[7 Gunluk Plan](#)[Konu Indeksi](#)[Internet Kaynaklari](#)[Quiz](#)[PDF Indir](#)[Panelleri ac](#)[Panelleri kapat](#)

# Haftalik Calisma Yolu — Ruby on Rails

Hafta 2'den 10'a kadar adim adim ilerle. Bilmiyorsan buradan basla; her adim seni dogru konuya goturur.

0 / 0 adim tamamlandi

Siradaki haftaya git

Sifirdan basliyorsan haftalari sirayla takip et. Her adimi tikla, sonra **Adima git** ile ilgili konuya gec.

2

## Hafta 2: Ruby & Rails Giris

+

2-3 saat

Ruby dil mantigini ve MVC yapisini anla; proje nasil calisir ogren.

3

## Hafta 3: ORM & Active Record

+

2-3 saat

SQL yazmadan veritabani islemi yap; find, find\_by, where farkini bil.

4

## Hafta 4: Enum, Routes & Controllers

+

2-3 saat

HTTP metotlarini ve 7 CRUD route'unu ogren.

5

## Hafta 5: Form & Strong Parameters

+

2 saat

Form ile veri al; guvenli kaydetme mantigini anla.

6

## Hafta 6: Bootstrap Frontend

+

2 saat

12 sutun grid ve navbar/footer yapisini ogren.

7

**Hafta 7: Model Iliskileri**

+

2-3 saat

belongs\_to / has\_many / HABTM iliskilerini ayirt et.

8

**Hafta 8: Validation & Active Storage**

+

2 saat

Model dogrulama kurallari ve dosya yukleme.

9

**Hafta 9: I18n, FriendlyId & Action Text**

+

2 saat

Coklu dil, SEO URL ve zengin metin editoru.

10

**Hafta 10: Devise & Authorization**

+

2-3 saat

Kullanici girisi ve rol bazli yetkilendirme.

# 7 Gun Kala — Sifirdan Gecer Not Plani

Ruby/Rails bilgin yoksa bu plani takip et. Her gun 2-3 saat yeterli. Gorevleri tiklayarak ilerlemeni kaydet.

0 / 0 gorev tamamlandi

Bugunun gorevine git

1

## Gun 1: Rails nedir? Temel iskelet

+

2-3 saat

MVC ve Ruby mantigini anla. Kod yazmana gerek yok.

2

## Gun 2: Veritabani & Active Record

+

2-3 saat

ORM ve find/find\_by/where farkini bil.

3

## Gun 3: Routes, HTTP & CRUD

+

2-3 saat

HTTP metotlari ve 7 CRUD route'u bil.

4

## Gun 4: Formlar & Guvenlik

+

2 saat

Strong Parameters ve form mantigini anla.

5

## Gun 5: Bootstrap & Model Iliskileri

+

2-3 saat

Grid sistemi ve tablo iliskilerini bil.

6

## Gun 6: Validation, I18n & Devise

+

2-3 saat

Dogrulama, coklu dil ve kullanıcı girişi konularını bitir.

7

### Gun 7: Tekrar & Sinav Gunu

+

3-4 saat

Tum test sorularını 2. kez coz. Ezber listesini tekrar et.

## Konu İndeksi — Alfabetik

Tablodaki **i** butonuna tıkla: terimin mantığı ve kullanımı acilir. **Kapsülleme, kalitim, polimorfizm** vurgulanmıştır.

| Konu                            | Hafta | Açıklama                          |
|---------------------------------|-------|-----------------------------------|
| Active Record                   | 3     | ORM kütüphanesi, model katmanı    |
| Active Storage                  | 8     | Dosya/resim yükleme               |
| Array (Dizi)                    | 2     | Ruby veri türü, metotları         |
| attr_accessor / reader / writer | 2     | <b>Kapsülleme</b> — getter/setter |
| Authentication                  | 10    | Kimlik doğrulama (Devise)         |
| Authorization                   | 10    | Yetkilendirme (admin?, roller)    |
| before_action                   | 5     | Controller callback (DRY)         |
| belongs_to                      | 7     | Model ilişkisi, FK bu tabloda     |
| Bloklar (each, map, yield)      | 2     | Ruby blok yapısı                  |
| Bootstrap Grid                  | 6     | 12 sütun responsive layout        |
| button_to                       | 5     | DELETE form butonu                |
| Constants (Sabitler)            | 2     | PI = 3.14, büyük harf             |
| CRUD                            | 4     | Create Read Update Delete         |
| CSRF / authenticity_token       | 5     | Form güvenlik tokeni              |
| Enum                            | 4     | Sayısal sabitlere isim (draft: 0) |
| ERB                             | 4     | Embedded Ruby, <% %> <%= %>       |
| FriendlyId                      | 9     | SEO slug URL                      |
| Global değişken (\$)            | 2     | \$monday — her yerden erişim      |
| Hash                            | 2     | Key-value Ruby yapısı             |
| has_many                        | 7     | Model ilişkisi                    |

| Konu                         | Hafta | Açıklama                        |
|------------------------------|-------|---------------------------------|
| has_many :through            | 7     | Ara tablo ile ilişki            |
| has_one                      | 7     | Bire-bir ilişki                 |
| HABTM                        | 7     | Çoktan çoğa, join table         |
| HTTP Metotları               | 4     | GET POST PUT PATCH DELETE       |
| l18n                         | 9     | Çoklu dil (t, l, locale)        |
| Instance variable (@)        | 2     | Nesneye ait değişken            |
| <b>Kalıtım (Miras)</b>       | 2     | class Dog < Animal, super       |
| <b>Kapsülleme</b>            | 2     | @name, getter/setter, attr_*    |
| Lambda vs Proc               | 2     | Argüman toleransı farkı         |
| Migration                    | 3     | Veritabanı şema değişikliği     |
| Modül (Module)               | 2     | include, namespacing            |
| MVC                          | 2     | Model View Controller           |
| Normalization                | 8     | Veriyi kaydetmeden düzenleme    |
| ORM                          | 3     | Tablo ↔ Sınıf eşlemesi          |
| Partial                      | 5     | _form.html.erb yeniden kullanım |
| Polimorfizm                  | 2     | speak metodunu override etme    |
| private / protected / public | 2     | Erişim kontrolü                 |
| Strong Parameters            | 5     | params.require.permit           |
| Symbol                       | 2     | :user tekil nesne               |
| Validation                   | 8     | validates kuralları             |
| Class variable (@@)          | 2     | Sınıfa ait değişken             |

## OOP Kavramları (Hafta 2 — Mutlaka Bil)

| Kavram            | Ruby'de Nasıl?                                        | Örnek                      |
|-------------------|-------------------------------------------------------|----------------------------|
| <b>Kapsülleme</b> | Instance variable @ ile gizle, getter/setter ile eriş | @name, attr_accessor :name |

| Kavram                 | Ruby'de Nasıl?                   | Örnek                                    |
|------------------------|----------------------------------|------------------------------------------|
| <b>Kalıtım (Miras)</b> | Alt sınıf üst sınıftan türetilir | <code>class Dog &lt; Animal</code>       |
| <b>Polimorfizm</b>     | Alt sınıf metodu override eder   | <code>def speak; "I'm a dog"; end</code> |
| <b>super</b>           | Üst sınıf metodunu çağırır       | <code>super + " from Cat class"</code>   |
| <b>Abstraction</b>     | Modül/sınıf ile arayüz tanımlama | <code>module Movable</code>              |



## Sınav Ezber Listesi — 15 Kritik Madde

7. gün ve sınav sabahı bu listeyi 2 kez oku. Çogu sınav sorusu bu maddelerden türetilir.

**Ruby'de her şey nesnedir**

3.class → Integer → Numeric → Object

**MVC akışı**

routes.rb → Controller → Model → View → HTML

**ORM**

Veritabanı tablosu = Ruby model sınıfı

**find(1)**

Bulamazsa exception (hata) fırlatır

**find\_by(...)**

Bulamazsa nil döner

**where(...)**

Filtreler, liste (Relation) döner

**resources :posts**

7 CRUD route otomatik oluşturur

**GET / POST / DELETE**

Oku / Oluştur / Sil

**Enum draft: 0**

DB'de 0, kodda draft anlamına gelir

**Strong Parameters**

Sadece izin verilen alanlar geçer (güvenlik)

**Partial \_form**

Dosya adı \_ ile başlar, tekrar kullanılır

**Bootstrap grid**

12 sütun; col-md-6 = yarım genişlik

**belongs\_to**

Foreign key bu tabloda bulunur

**Validation vs Normalization**

Validation reddeder, normalization düzenler

**Authentication vs Authorization**

Kimlik doğrulama vs yetki kontrolü

## Internet Kaynaklari ile Derinlestirilmis Konular

PDF slayt basliklari (kapsulleme, migration, kalitim vb.) resmi Ruby SSS ([ruby-lang.org](http://ruby-lang.org)) ve Rails dokumantasyonundan Turkce aciklamalarla desteklendi.

Tumu

Hafta 2

Hafta 3

Hafta 4

Hafta 5

Hafta 6

Hafta 7

Hafta 8

Hafta 9

Hafta 10

**H2** Kapsulleme (Encapsulation)

**H2** Kalitim / Miras (Inheritance)

**H2** Polimorfizm

**H2** MVC Mimarisi

**H2** Symbol vs String

**H3** ORM (Object Relational Mapping)

**H3** Migration

**H3** find / find\_by / where

**H4** Active Record Enum

**H4** HTTP Metotlari ve CRUD

**H4** ERB Sablonlari

**H5 Strong Parameters**

**H5 form\_with ve Partial**

**H6 Bootstrap Grid**

**H7 Model İlişkileri (belongs\_to / has\_many)**

**H7 HABTM (has\_and\_belongs\_to\_many)**

**H8 Validation (validates)**

**H8 Normalization (normalizes)**

**H8 Active Storage**

**H9 I18n (Yerelleştirme)**

**H9 FriendlyId (Slug URL)**

**H9 Action Text**

**H10 Devise (Authentication)**

**H10 Authorization (Yetkilendirme)**

**H2 Bloklar, Proc ve Lambda**

## Hafta 2 — Ruby & Rails Giriş

**Bu haftada ne öğreniyorsun?** Ruby dilini öğrenir, Rails projesini kurar ve MVC yapısını anlarsın.

Her şey nesne

MVC akisi

rails s

### Kısa Video Özeti (~1 dk) — Hafta 2



### Bu hafta — Ruby resmi SSS + internet açıklamaları (Türkçe):

Kapsülleme (Encapsulation)

Kalitim / Miras (Inheritance)

Polimorfizm

MVC Mimarisi

Symbol vs String

Bloklar, Proc ve Lambda

### Basit Anlatım

- Ruby'de sayı, metin, dizi — hepsi nesnedir.
- Rails = Model (veri) + View (arayüz) + Controller (mantık).
- routes.rb istegi yönlendirir, controller işler, view gösterir.
- rails new, db:create, rails s ile proje çalıştırılır.

## Rails blog uygulaması, postgresql veritabanı ve bootstrap kütüphanesi

–

kullanılarak aşağıdaki gibi oluşturulur. blogger-a-ogrencino Github’da oluşturulan repo ismiyle aynı olduğuna dikkat edin. Dikkat! ogrencino kısmı kendi öğrenci numaranız olacak.

Yukarıdaki kodu çalıştırdığınızda blogger-a-ogrencino adında bir rails projesini ilklendirmiş (başlatmış) olacaksınız.

### Ruby

+

### Ruby Veri Türleri – Array

+

### Ruby Veri Türleri – Array

+

### Ruby Veri Türleri – Array

+

### Ruby Veri Türleri – Array

+

### Ruby Veri Türleri – Float

+

### Ruby Veri Türleri – Hash

+

### Ruby Veri Türleri – Integer

+

### Ruby Veri Türleri – Integer

+

### Ruby Veri Türleri – String

+

Ruby Veri Türleri – String

+

Ruby Veri Türleri – String

+

Ruby Veri Türleri – Symbol

+

Rubygems

+

RubyGems, kütüphanelerin oluşturulması, paylaşılması ve kurulumu için tasarlanmış

+

Erişim Metotları Public, Private ve Protected

+

Metot Yazma Kuralları

+

Değişken Sayıda Argümanlar

+

Global Değişkenler ve Sabitler (Constants)

+

Local Değişkenler

+

Sınıf Değişkeni

+

Github Classroom Proje Depo Oluşturma Linki

+

Rails Veritabanı Kullanıcı Adı ve Parolayı Credentials’a Kaydetme

+

Sınıflar (Classes)

+

Son Kontroller

+

Modül (Module)

+

Modül (Module) Namespacing

+

Bloklar

+

Bloklar

+

Bloklar

+

Bloklar do/end ile ya da { } ile sarmalanmışlardır. Bloklar için each, map,

+

Veritabanı username ve password belirttikten sonra artık veritabanları

+

veritabanı username ve password bilgileri eklenmesi gerekmektedir.

+

Koşullar

+

Rails blog uygulamasında yapacağımız bütün geliştirmeler için artık bu

+

Rails Blog Uygulamasının Oluşturulması

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı – config

+

Rails Dizin Yapısı – config

+

Rails Dizin Yapısı – config

+

Rails Anasayfa Güncelleme

+

Rails MVC

+

Veritabanı şemasını nasıl değiştirileceğini tanımlayan

+

2. Hafta Ders İçeriği

+

Blog Uygulamasının Github'a Gönderilmesi

+



Blog Uygulamasının Github'a Gönderilmesi

+

Blog Uygulamasının Veritabanı Ayarı

+

Fonksiyonlar (Methods)

+

for Döngüsü

+

Kaynaklar

+

Proc (Procedures) ve Lambda

+

Proc (Procedures) ve Lambda Farkları

+

Rails uygulamasının

+

Rails Uygulamasının Çalıştırılması

+

ternary Operatörü

+

Veritabanı için ön tanımlı verilerin oluşturulduğu dosya

+

Veritabanı şeması

+

Veritabanı şifreleri, API anahtarları vs. gizli

+

## Hafta 3 — ORM & Active Record

**Bu haftada ne ogreniyorsun?** SQL yazmadan veritabani islemleri yaparsin. Tablo = Model sinifi.

ORM

find vs where

migration

Kisa Video Ozeti (~1 dk) — Hafta 3

...

**Bu hafta — Ruby resmi SSS + internet aciklamalari (Turkce):**

ORM (Object Relational Mapping)

Migration

find / find\_by / where

### Basit Anlatim

- ORM: veritabani tablosu = Ruby sinifi.
- Post.create ile kayıt ekle, Post.all ile listele.
- find id arar (hata verir), find\_by arar (nil doner), where filtreler.
- Migration ile tablo/kolon ekle; rails db:migrate calistir.

**Active Record, kalıcı depolama gerektiren Ruby nesnelerini bir veritabanına**

–

kaydetmenize ve kullanmanıza yardımcı olur.

Business Logic nedir?

Verilerin nasıl oluşturulacağını, depolanacağını ve değiştirileceğini belirleyen iş kurallarını kodlayan programın bir parçasıdır.

**ORM (Object Relational Mapping) Nedir?**

+

**ORM (Object Relational Mapping) Nedir?**

+

**ORM (Object Relational Mapping) Nedir?**

+

**ORM (Object Relational Mapping) Nedir?**

+

**ORM ile SQL sorgusu yazmadan veritabanına DDL (Data Definition Language)**

+

**ORM Kullanmanın Avantajları**

+

**Rails Migration Status**

+

**Rails Migration Status**

+

**Active Record'un Desteklediği Veri Türleri**

+

**Active Record'un Desteklediği Veri Türleri**

+

**Active Record'un Desteklediği Veri Türleri**

+

Active Record'un Desteklediği Veri Türleri

+

Migration dosyası ile yapılan değişiklikleri geri almak için rollback yapılır.

+

Migration sınıfları ActiveRecord::Migration sınıfını miras alır ve change adında

+

Rails Model Oluşturma

+

Veritabanındaki her bir tablo bir sınıfa karşılık gelmektedir.

+

Veri Tabanı Tablosu Oluşturma

+

Veri Tabanı Tablosu Oluşturma

+

Veri tabanına bağlanıp tabloları görüntüleyin:

+

Veritabanındaki posts tablosunun kolonlarını görüntüleyelim.

+

Konsolda Rails Modeliyle Etkileşim

+

Konsolda Rails Modeliyle Etkileşim

+

Konsolda Rails Modeliyle Etkileşim

+

Konsolda Rails Modeliyle Etkileşim

+

Veritabanından bir kayıt silmek için destroy kullanılır.

+

Where and Kullanımı

+

Where not ve Where or Kullanımı

+

Kayıtları Güncelleme

+

Kayıtları Güncelleme

+

Kayıtları güncellemenin 2 yolu var. Biri update kullanmak bir diğeri nesneye

+

Kayıtları Silmek

+

Migration bir veritabanı şemasının (schema) nasıl değiştirileceğini tanımlayan

+

Veritabanından Verilere Ulaşılması

+

Rails Migration Yapısı

+

Rails Migration Yapısı

+

Rails Migration Yapısı

+

Rails Migration Yapısı

+

Rails Migration Yapısı

+

Rails Migration Yapısı

+

3. Hafta Ders İçeriği

+

ActiveRecord Nedir?

+

ActiveRecord Nedir?

+

ActiveRecord::Relation

+

Genel

+

Post modelini oluşturduktan sonra uygun bir mesaj ile yapılanları Github'a

+

Spesifik Bir Kayıt Bulma

+

Spesifik Bir Kayıt Bulma

+

Spesifik Bir Kayıt Bulma

+

Tür Açıklama

+

Tür Açıklama

+

Tür Açıklama

+

Tür Açıklama

+

Veri tabanına kaydedildikten sonra post içeriği:

+

Veri tabanına kaydetmek için save etmeliyiz.

+

Veritabanı Üzerinde Yapılabilecek İşlemler

+

Veritabanı Üzerinde Yapılabilecek İşlemler

+

Veritabanındaki Kayıtlara Model Üzerinden Ulaşmak

+

Veritabanındaki Kayıtlara Model Üzerinden Ulaşmak

+

Veritabanındaki Kayıtlara Model Üzerinden Ulaşmak

+

Veritabanındaki Verileri Filtreleme

+

Veritabanındaki Verileri İstenen Sırada Getirme

+

## Hafta 4 — Enum, Routes, Controllers

**Bu haftada ne ogreniyorsun?** URL'leri controller'a baglarsin. 7 CRUD islemi resources ile gelir.

[HTTP metotlari](#)[resources :posts](#)[Enum](#)

### Kisa Video Ozeti (~1 dk) — Hafta 4



### Bu hafta — Ruby resmi SSS + internet aciklamalari (Turkce):

[Active Record Enum](#)[HTTP Metotlari ve CRUD](#)[ERB Sablonlari](#)

### Basit Anlatim

- GET okur, POST olusturur, PUT/PATCH gunceller, DELETE siler.
- resources :posts otomatik 7 route olusturur.
- Enum: sayisal degerlere anlamlı isim verir (draft: 0).
- @degisken controller'dan view'a veri tasir.



## ERB (Embedded Ruby) Nedir?

İçeriğimizin gösterileceği html dosyası views/posts/index.html.erb erb uzantılıdır.

ERB, ruby kodunu çalıştırarak Rails ile dinamik olarak HTML üretmemize olanak tanır.

### RUBY

```
<%= %> etiketi, ERB'ye içerideki Ruby kodunu çalıştırmasını ve dönüş değerini çıktı olarak vermesini söyler.  
<h1>Posts</h1>  
<% @posts.each do |post| %>  
<%= post.title %><br/>  
<%= post.article %><br/>  
<%= post.status %><br/>  
<% end %>  
views/posts/index.html.erb
```

### INTERNET KAYNAĞI + RUBY RESMİ SSS (TURKCE)

**ERB Sablonlari** — <% %> Ruby kodu çalıştırır, <%= %> çıktıyı HTML'e yazar.

ERB (Embedded Ruby) view dosyalarında Ruby kodu çalıştırır. <% if @post %> sadece çalıştırır, <%= @post.title %> HTML'e escape edilmiş çıktı yazar. <%- -%> boşluk kırpmaya için kullanılabilir.

#### RUBY RESMİ SSS (TURKCE) — BOLUM 5: ITERATOR'LAR

**S:** Blok nedir ve ERB ile ilişkisi?

**C:** Iterator, blok veya Proc kabul eden metottur; blok metod çağrısından hemen sonra yazılır. ERB sablonlari da gömülü Ruby kodu çalıştırır: <% %> kod çalıştırır, <%= %> sonucu yazar. View'daki each döngüleri FAQ'deki data.each { |i| puts i } örneğiyle aynı blok mantığını kullanır.

[ruby-lang.org](http://ruby-lang.org) resmi FAQ →

### ORNEK KOD

```
<%# yorum %>  
<h1><%= @post.title %></h1>  
<% @posts.each do |post| %>  
  <p><%= link_to post.title, post %></p>  
<% end %>
```

Index sayfasında sadece status durumu published postlar gösterilmesi için:

+

Post modeline status alanı ekleme

+

Post modeline status ekledikten sonra kolon bilgileri aşağıdaki gibi

+

Active Record Enum

+

Active Record Enum – Prefix

+

Active Record Enum – Suffix

+

Active Record Scope

+

Active Record Scope

+

Active Record Scope

+

Active Record Scopes

+

HTTP Metot ve Amaçları

+

HTTP Metot ve Amaçları

+

HTTP Metotlar ve Yollar (Routes)

+

Link Helpers

+

HTTP (Hyper Text Transfer Protocol)

+

http Protokol (Hyper-Text Transfer Protocol)

+

Rails Router

+

Rails Routes

+

Rails Routes

+

Rails Routes

+

Rails Routes

+

Rails Routes

+

Rails Routes – Resources

+

Veritabanına Seed Verisi Ekleme

+

Veritabanına Seed Verisi Ekleme

+

4. Hafta Ders İçeriği

+

Arayüzden Blog Girdisi Oluşturma

+

Belirli Bir Kaydın Görüntülenmesi

+

Belirli Bir Kaydın Görüntülenmesi

+

Index – Bütün post kayıtlarını gösterir.

+

Index Sayfa İçeriği

+

Post Modeli Sütun Bilgileri

+

URL Parçaları

+

Veritabanındaki bütün postları listeler.

+

Veritabanındaki verileri html sayfasında gösterelim.

+

## Hafta 5 — Form Helpers & Strong Parameters

**Bu haftada ne ogreniyorsun?** Form ile veri alir, Strong Parameters ile guvenli kaydedersin.

[form\\_with](#)[Strong Params](#)[partial](#)

Kisa Video Ozeti (~1 dk) — Hafta 5

⋮

**Bu hafta — Ruby resmi SSS + internet aciklamalari (Turkce):**

[Strong Parameters](#)[form\\_with ve Partials](#)

### Basit Anlatim

- `form_with` model: `@post` ile form olusturulur.
- `post_params` sadece izin verilen alanlari gecirir.
- `Partial` (`_form`) ile tekrar eden kod azaltilir.
- `before_action` ile ortak kod tek yerde toplanir.

## Action View

–

- MVC’de View’e karşılık gelir.
- Web isteklerini karşılamak için Action Controller ile birlikte çalışırlar.
- Action Controller, model katmanı ile iletişim kurmak ve veri almakla ilgilenir.
- Action View bu verileri kullanarak web isteğine bir yanıt gövdesi oluşturmaktan sorumludur.
- Action View, formlar, tarihler ve stringler için HTML etiketlerini dinamik olarak oluşturan bir çok yardımcı metot (helper) sağlar.

## Action View Form Helpers

+

## Action View Form Helpers

+

## Strong Parameter Nedir?

+

## Index Sayfasına Yeni Post Ekleme Linki Ekle

+

## Partials

+

## Partials

+

## Before Action Callback

+

## Before Action Callback

+

button\_to Kullanımı

+

Arayüzden Post Güncellemek

+

MVC İçeriklerinin Son Hali

+

Arayüzden Post Kaydını Silmek

+

5. Hafta Ders İçeriği

+

Arayüzden Post Oluşturmak

+

Kaynaklar

+

## Hafta 6 — Bootstrap Frontend

**Bu haftada ne ogreniyorsun?** Blog arayuzunu Bootstrap ile duzenlersin.

[12 sutun grid](#)[navbar/footer](#)[card](#)

Kisa Video Ozeti (~1 dk) — Hafta 6



**Bu hafta — Ruby resmi SSS + internet aciklamalari (Turkce):**

[Bootstrap Grid](#)

### Basit Anlatim

- Grid 12 sutun: col-md-6 = yarim genislik.
- container ortalar, container-fluid tam genislik.
- Navbar ve footer partial olarak layout'a eklenir.
- card, btn siniflari ile modern gorunum.



Hızlı ve kolay bir şekilde web uygulaması geliştirebilmek için kullanılan ücretsiz bir frontend çerçevesidir. Formlar, butonlar, tablolar, navigation, modals gibi html ve css templateleri mevcuttur ve isteğe bağlı bazı JS eklentileri de mevcuttur.

### INTERNET KAYNAĞI + RUBY RESMİ SSS (TURKCE)

**Bootstrap Grid** — 12 sütunluk responsive grid; container, row, col-\* sınıfları.

Bootstrap 5 grid sistemi 12 sütun üzerinden çalışır. container içeriği ortalar, row satır, col-md-6 medium ekranda yarım genişlik demektir. Rails 7+ ile rails new --css bootstrap veya importmap ile eklenir.

#### RUBY RESMİ SSS (TURKCE) — BOLUM 1: GENEL SORULAR

**S:** Ruby ile sunucu ve arayüz nasıl birlikte çalışır?

**C:** Bootstrap CSS/HTML katmanıdır; Ruby FAQ doğrudan Bootstrap anlatmaz. Ancak Rails view'lerinde ERB ile Ruby kodu çalıştırılırken HTML sınıfları (container, row, col) statik içerik olarak yazılır. Ruby'nin görevi veriyi hazırlamak (@posts), Bootstrap in görünümü düzenlemektir.

[ruby-lang.org resmi FAQ](https://ruby-lang.org/resmi-FAQ) →

#### ORNEK KOD

```
<div class="container">
  <div class="row">
    <div class="col-md-8">Ana içerik</div>
    <div class="col-md-4">Sidebar</div>
  </div>
</div>
```

[Ruby Resmi SSS — Bolum 1 \(Genel Sorular\)](#)

[Bootstrap Grid Docs](#)

**Bootstrap 5 mobil cihazlara duyarlı olacak şekilde tasarlanmıştır.**

+

**Bootstrap Breakpoints (Kırılma Noktaları)**

+

Bootstrap Container

+

Bootstrap Grid

+

Bootstrap Grid

+

Bootstrap Grid

+

Bootstrap ile Footer Ekleyelim

+

Bootstrap ile Navbar Ekleyelim

+

Bootstrap İnceleyin

+

Bootstrap JS

+

Bootstrap JS işlemleri düzgün çalışması için popper.js kütüphanesi

+

Form Sayfasını Güncelleyelim

+

Footer veya Navbar'a linkleyin. Linke basınca ilgili sayfa açılsın.

+

Hakkımda sayfası için path, view ve controller'da ilgili action'ı oluşturun.

+

Navbar'da görünmesini istediğiniz başka alanlar varsa link olarak

+

Projelerim sayfası için path, view ve controller'da ilgili action'ı oluşturun.

+

Footer için views/shared klasörü altında \_footer.html.erb adında partial

+

Navbar'daki Blogger yazısına (başka bir isim de verebilirsiniz) basınca

+

image\_tag HTML resim etiketi oluşturur. Resimler app/assets/images altına

+

image\_tag Kullanımı

+

image\_tag metodu ile kullanın.

+

Index Sayfasını Güncelleyelim

+

Index Sayfasını Güncelleyelim

+

Show Sayfasını Güncelleyelim

+

6. Hafta Ders İçeriği

+

Hakkımda

+

Hakkımda

+

Hakkımda Sayfası

+

Hakkımda Sayfası

+

Not

+

Projelerim Sayfası

+

Projelerim Sayfası

+

## Hafta 7 — Model İlişkileri & Kategori

**Bu haftada ne öğreniyorsun?** Tablolar arası bağlantı kurarsın. Post-Category çoktan coga.

belongs\_to/has\_many

HABTM

scaffold

Kisa Video Özeti (~1 dk) — Hafta 7



**Bu hafta — Ruby resmi SSS + internet açıklamaları (Türkçe):**

Model ilişkileri (belongs\_to / has\_many)

HABTM (has\_and\_belongs\_to\_many)

### Basit Anlatım

- belongs\_to: FK bu tabloda. has\_many: karşı taraf çok kayıt.
- HABTM: ara tablo, id yok, iki FK var.
- scaffold ile model+view+controller hızlı oluşur.
- category\_ids: [] ile çoklu kategori seçilir.

Form Sayfasına Kategori Select Box Ekleme

–

Rails Model ilişkileri – belongs\_to

+

Rails Model ilişkileri – has\_and\_belongs\_to\_many

+

Rails Model ilişkileri – has\_and\_belongs\_to\_many

+

Rails Model ilişkileri – has\_many ve belongs\_to

+

Rails Model ilişkileri – has\_one ve belongs\_to

+

kategorileri seçilebildi mi? (Strong Parameter 🌞

+

Rails Model ilişkileri – has\_many

+

Rails Model ilişkileri – has\_many :through

+

Rails Model ilişkileri – has\_many :through

+

Controller Güncelleme

+

Footer'a Kategoriler linki ekleyin.

+

Navbar'a İşlemler için dropdown menü ekleyelim. Yeni post ve kategori ekleme linkleri

+

Rails Model ilişkileri – has\_one

+

Rails Model ilişkileri – has\_one :through

+

Rails Model ilişkileri – has\_one :through

+

Rails Model ilişkileri – has\_one :through

+

Tablolar Arası Bağlantı Sağlayalım

+

Tablolar Arası Bağlantı Sağlayalım

+

Tablolar Arası Bağlantı Sağlayalım

+

Kategori görüntüleme sayfası örnek url: localhost:3000/categories/1

+

Kategori Sayfaları

+

Kategoriler Sayfasında İlgili Postları Listelemek

+

Post Girdisinin Kategorisini Belirleme

+

Post Show Sayfasına Kategoriler İçin Badge Ekleme

+

ActiveRecord::Migration[8.0]

+

7. Hafta Ders İçeriği

+

Rails Model ilişkileri

+

## Hafta 8 — Validation & Active Storage

**Bu haftada ne ogreniyorsun?** Veri kurallarini modelde tanimlarsin. Resim yuklemek icin Active Storage.

validates

normalizes

Active Storage

Kisa Video Ozeti (~1 dk) — Hafta 8



**Bu hafta — Ruby resmi SSS + internet aciklamalari (Turkce):**

Validation (validates)

Normalization (normalizes)

Active Storage

### Basit Anlatim

- validates: veriyi kontrol eder, gecersizse kaydetmez.
- normalizes: kaydetmeden once duzenler (bosluk sil, buyuk harf).
- presence, uniqueness, length en sik kullanan kurallar.
- has\_one\_attached :image ile dosya yukleme.



kullanır. Ancak :message seçeneği ile doğrulama başarısız olduğunda hatalar koleksiyonuna eklenecek mesajı kendimiz belirleyebiliriz.

### RUBY

```
class Post < ApplicationRecord
  validates :title, presence: { message: "Post başlığı boş bırakılamaz."}
end
```

### INTERNET KAYNAGI + RUBY RESMİ SSS (TURKCE)

**Validation (validates)** — Model kaydetmeden önce veri kurallarını kontrol eder.

validates :title, presence: true boş bırakılamaz der. uniqueness tekil olmayı, length uzunluk sınırını kontrol eder. Gecersiz kayıt save → false döner, errors mesajları dolar.

#### RUBY RESMİ SSS (TURKCE) — BOLUM 7: METOTLAR

**S:** Yıkıcı (destructive) metot nedir?

**C:** Yıkıcı metot nesnenin durumunu değiştirir. String'de str ve str! farkı gibi: validates kuralları geçmezse save false döner ve errors dolar — nesne durumu kaydedilmez. downcase! gibi ! ile biten metotlar alıcıyı yerinde değiştirir; validation geçersiz kaydı veritabanına yazmaz.

[ruby-lang.org resmi FAQ](https://ruby-lang.org/resmi-FAQ) →

### ORNEK KOD

```
class Post < ApplicationRecord
  validates :title, presence: true, length: { minimum: 3 }
  validates :slug, uniqueness: true
end

post = Post.new
post.valid? # => false
post.errors.full_messages
```

[Ruby Resmi SSS — Bolum 7 \(Metotlar\)](#)

[Rails Validations Guide](#)

Kategori modelindeki name alanının boş geçilmemesini ve normalizasyon

+

Kategori Validation ve Normalizasyon

+

Post Modeli Validation ve Normalizasyon Son Hali

+

Post modelinin title alanı için oluşturulan normalizasyon ile title bilgi

+

Rails Normalization

+

Rails Normalization

+

Rails Validation

+

Rails Validation

+

Validation Helpers

+

Validation Helpers – exclusion

+

Validation Helpers – format

+

Validation Helpers – inclusion

+

Validation Helpers – length

+

Validation Helpers – length

+

Validation Helpers – numericality

+

Validation Helpers – presence

+

Validation Helpers – uniqueness

+

Validation Helpers – uniqueness

+

Validation Options

+

Validation Options – allow\_blank

+

Validation Options – allow\_nil

+

Validation Options – if ve unless

+

Validation Options – message

+

Validation Options – on

+

Active Record validasyonları veritabanına eklenecek verilerin belirli kurallara

+

Validasyonları Atlayan Metotlar

+

Hata Mesajları

+

Hata Mesajları

+

Hata Mesajları

+

Sayfalarda Hata Mesajlarının Gösterimi

+

Sayfalarda Hata Mesajlarının Gösterimi

+

Sayfalarda Hata Mesajlarının Gösterimi

+

Active Storage

+

Active Storage – Dosya Attach Etme

+

Active Storage – Dosya Attach Etme

+

Active Storage, dosyaları Amazon S3, Google Cloud Storage veya Microsoft

+

8. Hafta Ders İçeriği

+

Post modelindeki title alanı hem benzersiz, hem boş geçilemez hem de

+

Tetikleyici (Trigger) Validasyonlar

+

veritabanına eklenir.

+

veritabanına kaydedilebilir.

+

veritabanına kaydedilmeden önce string'in başında, sonunda veya ortasında

+

## Hafta 9 — I18n, FriendlyId, Action Text

**Bu haftada ne öğreniyorsun?** Çoklu dil, SEO URL ve zengin metin editörü.

[I18n.t](#)[FriendlyId](#)[Action Text](#)

Kısa Video Özeti (~1 dk) — Hafta 9



**Bu hafta — Ruby resmi SSS + internet açıklamaları (Türkçe):**

[I18n \(Yerelleştirme\)](#)[FriendlyId \(Slug URL\)](#)[Action Text](#)

### Basit Anlatım

- I18n.t ile metin çevirirsin (tr.yml, en.yml).
- ?locale=en ile dil değiştirilir.
- FriendlyId: /posts/1 yerine /posts/baslik-slug.
- Action Text: zengin metin editörü (kalın, liste, resim).

Uygulamada diller arası geçiş nasıl yapılacaktır?

```
around_action :switch_locale
```

```
private
```

### RUBY

```
def switch_locale(&action)
  locale = params[:locale] || I18n.default_locale
  I18n.with_locale(locale, &action)
end

application_controller.rb
application_controller dosyasına yukarıdaki kodları ekleyelim.
```

Y erelleştirme Dosya Yapısı

config

locales

| -defaults

| ---tr.yml

| ---en.yml

| -models

| ---post

| -----tr.yml

| -----en.yml

| -views

| ---defaults

| -----tr.yml

| -----en.yml

| ---posts

| -----tr.yml

| -----en.yml

Y erelleştirmede Default Değerler

config/locales/defaults/tr.yml dosyası oluşturalım ve

<https://github.com/svenfuchs/rails-i18n/blob/master/rails/locale/tr.yml>

adresindeki içeriği kopyalayıp dosya içerisine kaydedelim.

config/locales/defaults/en.yml dosyası oluşturalım ve

<https://github.com/svenfuchs/rails-i18n/blob/master/rails/locale/en.yml>

adresindeki içeriği kopyalayıp dosya içerisine kaydedelim.

config/locales/defaults/tr.yml dosyasının en üstüne eylemler (actions)

için şunları ekleyin:

tr:

actions:

name: İşlemler

back: Geri

destroy: Sil

edit: Düzenle

new: Ekle

reset: Sıfırla

search: Ara

show: Görüntüle

update: Güncelle

Y erelleştirmede Default Değerler

config/locales/defaults/en.yml dosyasının içine eylemler (actions) için

şunları ekleyin:

en:

actions:

name: Actions

back: Back

destroy: Destroy

edit: Edit

new: New

reset: Reset

search: Search

show: Show

update: Update

Y erelleştirmede Default Değerler

Y erelleştirmelerin Sayfalarda Kullanımı

Bütün görüntüleme linklerini güncelleyelim. Örnek Post index sayfasındaki

show linki.

```
<%= link_to t("actions.show"), post_path(post), class:"btn btn-info
```

```
btn-sm" %>
```

## INTERNET KAYNAGI + RUBY RESMİ SSS (TURKCE)

**I18n (Yerelleştirme)** — Çoklu dil desteği; I18n.t ve config/locales/\*.yml dosyaları.

I18n modulu metinleri YAML dosyalarında tutar. I18n.t('posts.title') ceviri doner. URL'de ?locale=en veya session ile dil degistirilir. Rails varsayılan locale config/application.rb'de ayarlanır.

## RUBY RESMİ SSS (TÜRKÇE) – BÖLÜM 6: SOZDİZİMİ

**S:** Symbol sabit veya enum degeri olarak

**C:** FAQ: status = :open veya NORTH = :NORTH gibi Symbol'ler sabit veya enum degeri olarak kullanılabilir. I18n.t('posts.title') ceviri anahtarini string olarak alir; locale=:en gibi symbol parametreler dil seciminde kullanilir.

[ruby-lang.org](http://ruby-lang.org) resmi FAQ →

### ÖRNEK KOD

```
# config/locales/tr.yml
tr:
  posts:
    title: "Basliklar"

# view
<%= t('posts.title') %>
<%= link_to 'EN', url_for(locale: :en) %>
```

[Ruby Resmi SSS – Bölüm 6 \(Sozdizimi\)](#)

[Rails I18n Guide](#)

Internationalization (I18n) – Y erelleştirme

+

Internationalization (I18n) – Y erelleştirme

+

Yerelleştirme ayarlarının yapılması gerekmektedir. locale.rb dosyasını

+

Footer için eklenen yerelleştirmeleri kullanalım.

+

Footer'da gösterilen linkleri yerelleştirelim.

+

yerelleştirmelerini yapın.

+

Controller Düzenleme

+



Post show sayfasındaki edit, back ve destroy linklerini güncelleyelim.

+

footer:

+

footer:

+

Footer'a Türkiye ve ABD bayrak iconu kullanarak bunlara

+

Kategori sayfalarının post modelinde olduğu gibi bütün

+

Kategori Sayfalarının Y erelleştirilmesi

+

Kategori View Y erelleştirme

+

Kategori View Y erelleştirme

+

9. Hafta Ders İçeriği

+

activerecord:

+

activerecord:

+

activerecord:

+

activerecord:

+

Blog Girdilerinde Zengin İçerik Üretimi

+

Blog Girdilerinde Zengin İçerik Üretimi

+

Blog Girdilerinde Zengin İçerik Üretimi

+

Category Model Y erelleştirme

+

Category Model Y erelleştirme

+

Modele Ekleyelim

+

## Hafta 10 — Devise & Authorization

**Bu haftada ne ogreniyorsun?** Kullanici girisi ve rol bazli yetkilendirme.

[Devise](#)[authenticate\\_user!](#)[roller](#)

Kisa Video Ozeti (~1 dk) — Hafta 10

⋮

**Bu hafta — Ruby resmi SSS + internet aciklamalari (Turkce):**

[Devise \(Authentication\)](#)[Authorization \(Yetkilendirme\)](#)

### Basit Anlatim

- Devise: kayıt ol, giris yap, sifre sifirla.
- Authentication = kimlik, Authorization = yetki.
- `authenticate_user!` giris zorunlu kilar.
- `current_user.posts.new` ile post kullaniciya atanir.

devise :database\_authenticatable, :registerable,

:recoverable, :rememberable, :validatable,

:timeoutable

#### RUBY

end

app/models/user.rb

User modeline gender (cinsiyet) için parametreleri enum olarak ekleyelim ve

#### INTERNET KAYNAĞI + RUBY RESMİ SSS (TURKCE)

**Devise (Authentication)** — Hazır kullanıcı girişi: kayıt, giriş, şifre sıfırlama.

Devise gem'i User modeline moduller ekler (:database\_authenticatable, :registerable vb.).

authenticate\_user! giriş zorunlu kılar. current\_user oturum açmış kullanıcıyı döner. devise\_for

:users routes.rb'ye login/register yollarını ekler.

#### RUBY RESMİ SSS (TURKCE) — BOLUM 7: METOTLAR

**S:** Bu basit fonksiyon benzeri metotlar nereden geliyor?

**C:** class dışında yazılan def aslında Object sınıfına ait metottur; self gizli alıcıdır. Devise User modeline modul ekleyerek sign\_in, current\_user gibi metotları Controller'a getirir.

authenticate\_user! çağrılmazsa yönlendirme yapılır — Ruby'de her şey metot mesajıdır.

[ruby-lang.org resmi FAQ](https://ruby-lang.org/resmi-FAQ) →

#### ORNEK KOD

```
class ApplicationController < ActionController::Base
  before_action :authenticate_user!
end

class PostsController < ApplicationController
  def create
    @post = current_user.posts.new(post_params)
  end
end
```

[Ruby Resmi SSS — Bolum 7 \(Metotlar\)](#)

[Devise GitHub Wiki](#)

[Devise Getting Started](#)

devise :database\_authenticatable, :registerable,

+

Devise Gem Ekle

+

Devise Gem Ekle

+

Devise gem'ini Gemfile'e ekleyelim: <https://github.com/heartcombo/devise>

+

Devise için default çeviriler:

+

Devise Kütüphanesi

+

Devise Sayfalarını Düzenle

+

Devise Sayfalarını Düzenle

+

Devise Strong Parameter

+

Devise view sayfalarını oluşturun.

+

Devise, Rails uygulamaları için tam özellikli bir kimlik doğrulama (Authentication) çözümüdür.

+

devise\_parameter\_sanitizer.permit(:sign\_in, keys: [:email,

+

devise\_parameter\_sanitizer.permit(:sign\_up, keys: [:firstname,

+

Authentication

+

Authentication ve Authorization Farkları

+

Post için Strong Parameter

+

Navbar'ın güncel halini aşağıdaki linkteki koddan yardım alarak

+

Post Controller Düzenleme

+

Post modeline eklenen user\_id bilgisini controllers/posts\_controller.rb içindeki

+

Rol Ekleme

+

URL düzenlemesi için friendly\_id kütüphanesini kullanalım.

+

Admin Altında Kullanıcı Bilgileri

+

Admin Altında Kullanıcı Bilgileri

+

Admin Altında Kullanıcı Bilgileri

+

Admin Altında Kullanıcı Bilgileri

+

Admin Altında Kullanıcı Sayfaları

+

Navbar Güncelle

+

Navbar Güncelle

+

navbar'da gözükecek.

+

Post ile User Arasında İlişkileri Ekleyelim

+

Migration dosya içerisinde eklediğimiz bazı alanların düzenlenmesi:

+

activerecord:

+

activerecord:

+

Genel

+

kaynaklara veya işlemlere erişim hakkı

+

Kullanıcı Kaydı Gerçekleştirin

+

Post Modeline Kullanıcı Bilgisi Ekleme

+

## Kendini Test Et — 39 Soru

Soruya tikla, cevabi gor. Sinav oncesi tum sorulari en az bir kez coz.

### 1 Ruby'de 3.class.superclass.superclass ne doner?

Object doner. Hiyerarsi: Integer → Numeric → Object → BasicObject. Yani 3 bir Integer nesnesidir ve ust sinifi Numeric, onun ust sinifi Object'tir.

### 2 Symbol ile String arasindaki temel fark nedir?

Symbol (:user) tekil bir nesnedir; ayni sembol her zaman ayni object\_id'ye sahiptir. String her olusturulduunda yeni bir nesne yaratir. Semboller genelde hash key ve sabit isimler icin kullanilir.

### 3 attr\_accessor, attr\_reader, attr\_writer farklari?

attr\_reader: sadece okuma (getter). attr\_writer: sadece yazma (setter). attr\_accessor: hem getter hem setter olusturur. Ornek: attr\_accessor :name ile hem name hem name= kullanilabilir.

### 4 Proc ile Lambda arasindaki arguman farki?

Proc fazla veya eksik argumanda hata vermez (eksigi nil sayar, fazlasini yok sayar). Lambda arguman sayisina katidir; yanlis sayida arguman verilirse ArgumentError olur.

### 5 ? ve ! ile biten metotlarin anlami?

? ile biten metotlar boolean doner (even?, zero?). ! ile biten metotlar destructive/tehlikeli islem yapar; orijinal veriyi degistirir (upcase! gibi).

### 6 ORM'in avantajlari nelerdir?

Daha az SQL yazilir, nesne yonelimli calisilir, veritabani platformundan bagimsizlik saglanir (PostgreSQL'den MySQL'e gecis kolay), modelleme uygulama tarafinda yapilir, bakim maliyeti dusuktur.

### 7 find, find\_by ve where arasindaki fark?

find(id): id ile arar, bulamazsa exception firlatir. find\_by(...): tek kayit veya nil doner. where(...): sifir veya cok kayit donen ActiveRecord::Relation nesnesi doner.



**8** save ile create arasindaki fark?

Post.new ile once nesne olusturulur, save ile kaydedilir (iki adim). Post.create ile nesne olusturulur ve aninda veritabanina yazilir (tek adim).

**9** db:setup ile db:reset ne yapar?

db:setup = schema:load + seed (sema yuklenir, seed verisi eklenir). db:reset = db:drop + setup (veritabani silinir, sifirdan olusturulur ve seed eklenir).

**10** Migration rollback nasil yapilir?

rails db:rollback komutu son migration'i geri alir. rails db:migrate:status ile durum kontrol edilir. STEP parametresi ile birden fazla migration geri alınabilir.

**11** GET ve POST ne zaman kullanilir?

GET: veri okumak icin (sayfa gosterme, listeleme). POST: yeni kayıt olusturmak icin (form gonderme). GET ile hassas veri gonderilmemeli.

**12** PUT ile PATCH farki?

PUT kaynagin tum alanlarini gunceller. PATCH sadece degisen alanlari gunceller (kismi guncelleme). Ikisi de mevcut kaydi guncellemek icin kullanilir.

**13** resources :posts kac route olusturur? Isimleri?

7 CRUD route: index, show, new, create, edit, update, destroy. Ornek: GET /posts (index), GET /posts/:id (show), POST /posts (create), DELETE /posts/:id (destroy).

**14** posts\_path ile posts\_url farki?

posts\_path sadece yol doner: /posts. posts\_url tam URL doner: http://localhost:3000/posts (protokol + host + port dahil).

**15** Enum'da draft: 0 ne anlama gelir?

Veritabaninda status alanı 0 olarak saklanır, Ruby tarafında draft anlamına gelir. Yeni kayıt default 0 ise otomatik draft olur. post.draft? ve Post.draft scope kullanılabilir.

#### 16 Strong Parameters neden gerekli?

Mass assignment saldırısını onler. Sadece izin verilen alanlar (permit) modele aktarılır. Örnek: `params.require(:post).permit(:title, :article)` — baska alanlar yok sayılır.

#### 17 authenticity\_token ne ise yarar?

CSRF (Cross-Site Request Forgery) koruması sağlar. Formda hidden field olarak gönderilir, Rails oturumdaki token ile karşılaştırır. Eşleşmezse istek reddedilir.

#### 18 Partial dosya adlandırma kuralı?

Partial dosya adı `_` ile başlar: `_form.html.erb`. `render 'form'` veya `render partial: 'form'` ile çağrılır. Aynı formu new ve edit sayfalarında tekrar kullanmak için idealdir.

#### 19 button\_to ile link\_to farkı (DELETE için)?

`link_to` sadece link oluşturur; DELETE için ek ayar gerekir. `button_to` kendi mini formunu oluşturur, `_method=delete` hidden field ekler — silme işlemi için daha güvenli ve standart.

#### 20 before\_action ne sağlar?

DRY prensibi: tekrarlayan kodu action'lardan önce çalıştırır. Örnek: `before_action :set_post, only: [:show, :edit, :update]` — `@post` her action'da otomatik yüklenir.

#### 21 Bootstrap grid kaç sütundur?

12 sütunludur. Örnek: `col-4 + col-4 + col-4 = 12`, yan yana 3 eşit sütun. `col-8 + col-4 = 12`, geniş + dar sütun.

#### 22 .container ile .container-fluid farkı?

`container`: ekran genişliğine göre max-width sınırlı (ortalı). `container-fluid`: her zaman %100 genişlik, sınır yok.

#### 23 col-md-8 ne zaman yan yana, ne zaman alt alta?

768px ve üzeri ekranlarda yan yana durur. 768px altında (mobil) sütunlar otomatik üst üste yığılır (responsive).

**24** belongs\_to FK hangi tabloda?

Foreign key, belongs\_to tanımlanan modelin tablosundadır. Örnek: Book belongs\_to :author → books tablosunda author\_id sütunu vardır.

**25** has\_many :through ne zaman kullanılır?

İki model arasında ara (join) tablo olduğunda. Örnek: Physician has\_many :patients, through: :appointments — randevu tablosu üzerinden doktor-hasta ilişkisi.

**26** HABTM join table özelliği?

id sütunu yoktur (id: false). Sadece iki foreign key içerir (post\_id, category\_id). Post ve Category çoktan çokya bağlanır. Örnek: categories\_posts tablosu.

**27** dependent: :destroy ne yapar?

Üst kayıt silindiğinde bağlı alt kayıtları da otomatik siler. Örnek: Author silince has\_many :books, dependent: :destroy ile kitapları da silinir.

**28** Validation ile Normalization farkı?

Validation veriyi kontrol eder, geçersizse reddeder (save başarısız). Normalization veriyi kaydetmeden önce değiştirir (squish, titlecase) — reddetmez, düzenler.

**29** create! ile create farkı?

create başarısız olursa false döner veya hatalı nesne döner. create! başarısız olursa exception fırlatır (ActiveRecord::RecordInvalid). ! isareti exception garantisi verir.

**30** validates :title, presence: true, uniqueness: true, length: { maximum: 255 } ne kontrol eder?

Başlık boş olamaz, benzersiz olmalı (aynı başlıkla ikinci kayıt olamaz) ve en fazla 255 karakter olabilir.

**31** HTTP 422 ne anlama gelir?

Unprocessable Entity — sunucu isteği anladı ama validasyon hataları nedeniyle işleyemedi. render :new, status: :unprocessable\_entity ile form hataları gösterilir.

**32** l18n.t ve l18n.l farkı?

`l18n.t` (translate): metin çevirir — buton, label, mesaj. `l18n.l` (localize): tarih ve saat nesnelerini yerel formata çevirir. Örnek: `t('actions.show')`, `l(Time.now)`.

### 33 FriendlyId ne sağlar?

URL'de id yerine okunabilir slug kullanır. `/posts/1` yerine `/posts/ruby-programlamaya-giris`. SEO için faydalıdır. `friendly_id :title, use: :slugged`

### 34 Action Text ne için kullanılır?

Zengin metin editörü (WYSIWYG) sağlar — kalın, italik, liste, resim ekleme. `has_rich_text :article` ve `form.rich_textarea :article` ile kullanılır.

### 35 Authentication ile Authorization farkı?

Authentication (kimlik doğrulama): Kullanıcı kim? — giriş yapma. Authorization (yetkilendirme): Ne yapabilir? — giriş sonrası erişim hakkı. Devise authentication, Pundit/CanCanCan authorization için.

### 36 `current_user` ve `user_signed_in?` ne döner?

`current_user`: giriş yapmış User nesnesini döner (yoksa nil). `user_signed_in?`: boolean — true ise oturum açık, false ise açık değil.

### 37 `authenticate_user!` ne yapar?

Giris yapılmamışsa kullanıcıyı login sayfasına yönlendirir. `before_action :authenticate_user!` ile korumalı sayfalar oluşturulur.

### 38 Admin namespace'te yetki kontrolü nasıl yapılır?

`before_action :authorize_admin` tanımlanır. `unless current_user.admin?` ise `root_path`'e redirect edilir. Sadece admin rolundekiler `admin/users` sayfalarına erişebilir.

### 39 `current_user.posts.new` neden kullanılır?

Post otomatik olarak giriş yapan kullanıcıya atanır. `user_id` elle göndermek yerine ilişki üzerinden oluşturma güvenli ve doğru yöntemdir.

## Komut Hizli Referans

Terminalde en cok kullanacagin Rails komutlari. Ezber listesi olarak kullan.

### BASH

```
rails new app -d postgresql --css bootstrap
rails s / rails c
rails g model Post title:text
rails g migration AddStatusToPost status:integer
rails db:create / db:migrate / db:rollback / db:seed / db:reset
rails routes
rails active_storage:install
bin/rails action_text:install
bundle add devise
rails generate devise:install
git add . && git commit -m "mesaj" && git push
```



## Nesne Yonelimli Programlama

NYP II — Hafta 1-9 PDF arsivi. Hafta 10: SOLID calismalari ve cozumlu alistirmalar.

[Haftalik Yol](#)[Konu Indeksi](#)[Hafta 10 SOLID](#)[OOP Ezber](#)

OOP ara: solid, kalitim, kapsulleme...

# Haftalik Calisma Yolu — Python OOP

Hafta 1'den 10'a kadar adim adim ilerle. Her haftayi ac, gorevleri tikla, konulara gec.

0 / 0 adim tamamlandi

Siradaki haftaya git

Sifirdan basliyorsan haftalari sirayla takip et. Her adimi tikla, sonra **Adima git** ile ilgili konuya gec.

1

## Hafta 1: OOP Giris & Prosedurel Programlama

+

2 saat

Prosedurel ile OOP farkini anla; Python'da class mantigina gir.

2

## Hafta 2: Siniflar & Nesneler

+

2-3 saat

class, \_\_init\_\_, self ve instance variable kavramlari.

3

## Hafta 3: Kapsulleme (Encapsulation)

+

2 saat

Veriyi gizleme, getter/setter, @property.

4

## Hafta 4: Kalitim (Inheritance)

+

2-3 saat

class Child(Parent), override ve super().

5

## Hafta 5: super, Coklu Miras & @staticmethod

+

2-3 saat

MRO sirasi ve staticmethod farki.

6

## Hafta 6: Polimorfizm & Soyut Sinif

+

2-3 saat

Polimorfizm, duck typing, ABC.

7

### Hafta 7: Tasarim Desenleri — Giris

+

2 saat

Singleton, Factory gibi temel desenler.

8

### Hafta 8: Tasarim Desenleri — Devam

+

2 saat

Observer, Decorator desenleri.

9

### Hafta 9: SOLID Prensipleri (PDF)

+

2-3 saat

SRP, OCP, LSP, ISP, DIP tanimlarini ogren.

10

### Hafta 10: SOLID Calisma & Odev Cozumleri

+

3-4 saat

Odev formatinda SOLID; sinif arkadas ornekleri ve PDF alistirmalari.

## OOP Sınav Ezber Listesi

Nesne Yönelimli Programlama sınavından önce bu 9 maddeyi iki kez oku.

### Kapsulleme

@private veri + getter/setter veya attr\_accessor ile kontrollü erişim

### Kalitim

class Child < Parent — alt sınıf üst sınıftan özellik alır

### Polimorfizm

Aynı metod adı, farklı sınıflarda farklı davranış (override)

### Soyutlama

Gereksiz detayı gizle; abstract class / interface ile sözleşme

### LSP

Alt sınıf üst sınıfın yerine konulunca program bozulmamalı

### SRP

Bir sınıf tek sorumluluk — tek değişim nedeni

### OCP

Genişlemeye açık, değişikliğe kapalı

### ISP

Kullanılmayan arayüze bağımlı olma

### DIP

Somut sınıf yerine soyutlamaya bağımlı ol



## OOP Konu İndeksi

Tablodaki **i** butonuna tikla: terimin ne olduğu, nasıl kullanıldığı ve örnek kod acilir.

| Konu              | Hafta | Açıklama                                     |
|-------------------|-------|----------------------------------------------|
| Prosedürel vs OOP | 1     | Fonksiyon merkezli vs nesne merkezli         |
| Sınıf & Nesne     | 2     | class, <b>init</b> , self, instance variable |
| Kapsülleme        | 3     | private, getter/setter, @property            |
| Kalıtım           | 4     | class Child(Parent), override                |
| super()           | 5     | Üst sınıf constructor/metot çağırısı         |
| Çoklu miras & MRO | 5     | Birden fazla üst sınıf                       |
| @staticmethod     | 5     | Nesneye bağlı olmayan metot                  |
| Polimorfizm       | 6     | Aynı arayüz, farklı davranış                 |
| Soyut sınıf (ABC) | 6     | @abstractmethod                              |
| Tasarım desenleri | 7–8   | Singleton, Factory, Observer vb.             |
| <b>SOLID</b>      | 9     | S, O, L, I, D ilkeleri                       |
| SRP               | 9     | Tek sorumluluk                               |
| OCP               | 9     | Açık/kapalı — AlanHesaplayıcı alıştırmaları  |
| LSP               | 9     | Dosya sistemi alıştırmaları                  |
| ISP               | 9     | Akıllı ev cihazları alıştırmaları            |
| DIP               | 9     | Bildirim sistemi alıştırmaları               |

## Hafta 1 — OOP Giriş & Prosedürel Programlama

**Bu haftada ne öğreniyorsun?** Prosedurel vs OOP, ders kurallari, OOP tarihi ve temel prensipler.

Prosedurel = fonksiyon + global veri

OOP = nesne (veri + davranis)

Python'da class ile baslariz

### Basit Anlatim

- Prosedurel = fonksiyon + global veri
- OOP = nesne (veri + davranis)
- Python'da class ile baslariz

#### 1. Hafta

–

Öğr. Gör. Ecmel Albayrak

#### 1. Hafta

+

#### Dersin Kuralları

+

#### Beklentiler

+

#### Dersin Amacı

+

#### Nesne Yönelimli Programlama (Object

+

#### Ders Kaynakları (Kitap)

+

Teams Katılım Kodu

+

Geliştirme Ortamı

+

Prosedürel Programlama

+

Prosedürel Programlamada Temel Mantık

+

Prosedürel programlamada önemli

+

Prosedürel Programlamanın Temel Yapı Taşları

+

Prosedürel yaklaşımda:

+

global değişken)

+

Global verilere her fonksiyon

+

Fonksiyonlar var ama bir

+

Prosedürel Programlamanın Avantajları

+

Prosedürel programlama kötü değildir, aksine:

+

Prosedürel Programlamanın Dezavantajları

+

Prosedürel Programlamanın Dezavantajları

+

Prosedürel Programlamanın Dezavantajları

+

fonksiyonların da

+

Prosedürel Programlamanın Dezavantajları

+

Prosedürel Programlamanın Dezavantajları

+

Prosedürel Programlamanın Dezavantajları

+

Prosedürel Programlamanın Dezavantajları

+

Prosedürel Programlamanın Dezavantajları

+

Prosedürel programlama küçük işler için yeterlidir, ancak yazılım

+

Nesne Yönelimli Programlama İhtiyacı

+

Prosedürel vs Nesne Yönelimli

+

Prosedürel Nesne Yönelimli

+

Fonksiyon odaklı Nesne odaklı

+

Nesne Yönelimli Programlama Tarihi

+

Nesne Yönelimli Programlama (Object Oriented Programming) Nedir?

+

Sınıf, nesnelerin özelliklerini ve işlevlerini (davranışlarını) tanımlamak için

+

OOP Sınıf Kavramı

+

OOP Sınıf Kavramı

+

Sınıf planından üretilmiş gerçek/somut varlıklardır. Planı kullanarak

+

OOP Nesne Kavramı

+

OOP Nesne Kavramı

+

OOP Özellik Kavramı

+

OOP Özellik Kavramı

+

OOP Metot Kavramı

+

OOP Metot Kavramı

+

Miras Alma (Inheritance)

+

Kapsülleme (Encapsulation)

+

Soyutlama (Abstraction)

+

sınıfın özellikler

+

Miras Alma (Inheritance)

+

Miras Alma Ne Sağlar

+

Kapsülleme (Encapsulation)

+

Kapsülleme Ne Sağlar

+

Soyutlama (Abstraction)

+

Soyutlama Ne Sağlar

+

Prosedürel Programlama Temelleri

+

Nesne Yönelimli Programlama Temelleri

+

## Hafta 2 — Sınıflar & Nesneler

**Bu haftada ne öğreniyorsun?** class, \_\_init\_\_, self, instance variable ve instance metotlar.

class = kalıp, nesne = örnek

self = bu nesne

\_\_init\_\_ otomatik çalışır

### Basit Anlatım

- class = kalıp, nesne = örnek
- self = bu nesne
- \_\_init\_\_ otomatik çalışır

### 2. Hafta

–

Öğr. Gör. Ecmel Albayrak

### 2. Hafta

+

Sınıf, nesnelerin nasıl oluşturulacağını tanımlayan bir şablondur.

+

Sınıf → Plan / Kalıp

+

Nesne → Üretilen ürün

+

Sınıf, sadece tarif eder. Nesne, bir sınıftan oluşturulan gerçek bir

+

Sınıflar

+

Sınıf isimleri genellikle büyük harfle başlar (CamelCase).

+

Sınıf Tanımlama

+

Sınıf adını parantezlerle çağırarak nesne oluşturulur.

+

Nesne Tanımlama

+

Nesne oluşturulduğu anda otomatik çalışan metottur.

+

`__init__` Metodu: Yapıcı Metot (Constructor)

+

`__init__` çalışır.

+

Instance (Örnek) Değişken Nedir?

+

`self` Parametresi Nedir?

+

Self olmazsa, değişken nesneye ait olmaz.

+

`self` Parametresi Nedir?

+

`self` sayesinde:

+

Instance (Örnek) Metotlar – Nesne Davranışı

+

Metotların Kullanımı

+



self Olmadan Hata

+

Nesne durumu (state), bir nesnenin o

+

Sınıfı odunc\_al ve

+

Sınıfı kitap\_oku

+

Sınıfı

+

nesne üzerinde kullanımı

+

Sınıf ve Nesne Oluşturma

+

Instance (Örnek) Metotları Kullanma

+

## Hafta 3 — Kapsülleme

**Bu haftada ne öğreniyorsun?** Veriyi gizleme, getter/setter, @property, private convention.

\_ veya \_\_ ile gizle

@property ile kontrollü erişim

Encapsulation = güvenli veri

### Basit Anlatım

- \_ veya \_\_ ile gizle
- @property ile kontrollü erişim
- Encapsulation = güvenli veri

Genel

–

||

3. Hafta

+

3. Hafta

+

Kapsülleme, bir nesnenin iç verilerini doğrudan erişime kapatarak, bu

+

Kapsülleme

+

Kapsülleme

+

Kapsülleme yalnızca değişkenleri private veya protected yaparak dış

+

Kapsülleme Gerçek Hayat Örneği

+

Kapsülleme Yoksa Ne Olur?

+

Kapsülleme Yoksa Ne Olur?

+

Kapsülleme olmadığında

+

Kapsülleme Yoksa Ne Olur?

+

Kapsülleme Yoksa Ne Olur?

+

Sınıf üyelerine (özellikler ve metotlar) nereden erişilebileceğini kontrol

+

sınıflardan bile erişilebilir. Her yerden erişime açık.

+

sınıflardan doğrudan erişilemez.

+

Python'da özellik gibi erişip aslında metot çalıştırmak mümkündür.

+

property ile getter metot

+

metotdur. Getter metot veri

+

property ile setter metot

+

Getter metot adı attribute ile aynı olmalı.

+

Property ile davranış/metot yapılmaz. Property yalnızca veri içindir.

+

Metot ile davranış sağlanmalı

+

Property ve normal metot aynı isimde olamaz

+

Kapsülleme ile İlgili Önemli Hususlar – Özet

+

Kapsülleme ile İlgili Önemli Hususlar – Özet

+

Kapsülleme ile İlgili Önemli Hususlar – Özet

+

sınıfın kendi içinde

+

Sınıfın iç işleyişinde kullanılan

+

Kapsülleme (Encapsulation) Çalışma

+

Prensibi

+

## Hafta 4 – Kalitim (Miras)

**Bu haftada ne ogreniyorsun?** class Child(Parent), method overriding, isinstance.

Kalitim = kod tekrarini azaltir

Override = ust metodu yeniden yaz

super() ile ust cagir

### Basit Anlatim

- Kalitim = kod tekrarini azaltir
- Override = ust metodu yeniden yaz
- super() ile ust cagir

#### 4. Hafta

–

Öğr . Gör. Ecmel Albayrak

#### 4. Hafta

+

#### Miras Alma (Inheritance)

+

#### Sınıflarda is-a ve has-a ilişkisi

+

#### Sınıf Özellikleri

+

#### Sınıf Metotları

+

#### Miraslama, bir sınıfın başka bir sınıfın özelliklerini (veri)

+

Miraslama

+

Miraslamaya Neden İhtiyaç Duyarız?

+

Miraslamaya Neden İhtiyaç Duyarız?

+

Miraslama olmazsa, her sınıf

+

Miraslamaya Neden İhtiyaç Duyarız?

+

\_\_init\_\_ metodunu Kedi ve Kopek

+

sınıfı için 2 kere yazdık (isim, yas

+

Miraslama sayesinde gerçek dünyadaki "A, B'dir" ilişkisini kurabiliriz.

+

Miraslama Nasıl Yapılır?

+

sınıfından miras alır.

+

Miraslama Nasıl Yapılır?

+

sınıfından miraslama

+

Miras alınan üst sınıfları ayırabiliriz: KaraHayvanları ve DenizHayvanları

+

metotlarını bilmiyor,

+

bağımlılık tek yönlü

+

Nesne özelliği: Her nesnede farklı (self.isim, self.yas)

+

Sınıf özelliği: Tüm nesnelerde ORTAK (Hayvan.tur)

+

Sınıf Özellikleri (Class Attribute)

+

Sınıf Özellikleri (Class Attribute)

+

Sınıf metotları, bir nesneye değil, sınıfın kendisine ait davranışları

+

Sınıf Metotları (Class Methods)

+

Sınıf özellikleri (class attributes) yönetiliyorsa

+

self üzerinden veri kullanmak

+

Nesne davranışlarını tanımlamak

+

Sınıf Özelliği ve Sınıf Metot Kullanımı

+

Sınıf özelliği gerekir

+

Sınıf Özelliği ve Sınıf Metot Kullanımı

+

Sınıf Özelliği ve Sınıf Metot Kullanımı

+

Sınıf Özelliği ve Sınıf Metot Kullanımı

+

self → nesne

+

Sınıf Özelliği ve Sınıf Metot Kullanımı

+

Miras Alma

+

Sınıf Özelliklerinin Kullanımı

+

Sınıf Metotlarının Kullanımı

+



## Hafta 5 — super, Çoklu Miras, Statik Metot

**Bu haftada ne öğreniyorsun?** super(), çoklu miras, MRO sırası, @staticmethod.

super().\_\_init\_\_() üst constructor

MRO = metot arama sırası

@staticmethod nesne gerektirmez

### Basit Anlatım

- super().\_\_init\_\_() üst constructor
- MRO = metot arama sırası
- @staticmethod nesne gerektirmez

5. Hafta

–

5. Hafta

+

Çoklu Miraslama

+

Miraslama

+

sınıfta tekrar yazmak zorunda

+

sınıf), ben sadece cins ekliyorum”

+

sınıfının constructor’ını çağırır

+

Metotlarda super() Kullanımı

+

Çoklu Miraslama

+

Çoklu Miraslama

+

Çoklu Miraslama – Base ve Alt Sınıflar

+

Çoklu Miraslama – MRO

+

super kullanmazsan ne olur? Aynı init iki kere çalışır.

+

super kullanmazsan ne olur?

+

Çoklu Miraslamada Metot Kullanımı

+

sınıfların metotlarını

+

Çoklu Miraslamada Metot Çakışması

+

Çoklu Miraslamada Manuel Çözüm

+

Çoklu Miraslamada super ile Zincir Yapısı

+

Çoklu Miraslamada super ile Zincir Yapısı

+

Çoklu Miraslamada super ile Zincir Yapısı

+

Static Metot (@staticmethod) Nedir?

+

Static metot, bir sınıfa ait olan ancak ne nesneye (self) ne de sınıfa

+

Static Metot (@staticmethod) Nedir?

+

Nesne oluşturmaya gerek yok, self yok, cls yok

+

Static metot sınıfa aittir, sınıf üzerinden çağırılması daha doğrudur.

+

Static Metot ile Nesne Metodu Karşılaştırma

+

Nesne metodu: self alır, nesneye bağlıdır, nesne durumu ile çalışır

+

Static Metot ile Nesne Metodu Karşılaştırma

+

Static metot nesne üzerinden çağırılabilir ama nesneyi kullanmaz,

+

self gönderilmez

+

Miras Almada super() Kullanımı

+

Çoklu Miraslama

+

Static Metotlar

+

## Hafta 6 — Polimorfizm & Soyut Sınıf

**Bu haftada ne öğreniyorsun?** Polimorfizm, duck typing, ABC ve @abstractmethod.

Aynı metod farklı davranış

ABC'den doğrudan nesne yok

Duck typing = ne yaptığına bak

### Basit Anlatım

- Aynı metod farklı davranış
- ABC'den doğrudan nesne yok
- Duck typing = ne yaptığına bak

Genel

–

II

6. Hafta

+

6. Hafta

+

Duck Typing

+

Polimorfizm Olmazsa Ne Olur?

+

Polimorfizm Olmazsa Ne Olur?

+

Polimorfizm Olmazsa Ne Olur?

+

Sınıf nesneleri oluşturulur ve her nesne için üretilen metotlar kullanılır.

+

Polimorfizm Olmazsa Ne Olur?

+

Polimorfizm Olmazsa Ne Olur?

+

sınıf eklediğimizde

+

Polimorfizm ile Çözüm

+

Metot Ezme (Overriding): Kalıtım ilişkisinde, alt sınıfın üst sınıftaki bir metodu kendi

+

Polimorfizm ile Çözüm

+

Polimorfizm ile Çözüm

+

Polimorfizm ile Çözüm

+

SOLID – Open/Closed Principle

+

Polimorfizm Çözümünün Özellikleri

+

Polimorfizm Faydaları

+

KODU GENİŞLETİLEBİLİR YAPAR (Open/Closed Prensipleri)

+

KOD TEKRARINI ÖNLER (DRY Prensipleri)

+

Polimorfizm Faydaları

+

Duck Typing Nedir?

+

Duck Typing Nedir?

+

Kalıtıma gerek yoktur.

+

Duck Typing Ne Zaman Kullanılmalı

+

Duck Typing

+

Duck Typing Örneği

+

Duck Typing Örneği

+

Polimorfizm ve Kompozisyon (has-a) Birlikte

+

Polimorfizm için ortak arayüz

+

Polimorfizm ve Kompozisyon Birlikte

+

Polimorfizm ve Kompozisyon Birlikte

+

Polimorfizm ve Kompozisyon Birlikte

+

Polimorfizm Çalışma Sorusu – 1

+

Polimorfizm Çalışma Sorusu – 1 (Örnek Çıktı)

+

Polimorfizm Çalışma Sorusu – 2

+

Polimorfizm Çalışma Sorusu – 2

+

Polimorfizm Çalışma Sorusu – 2

+

Polimorfizm nedir?

+

Polimorfizmin faydaları neler?

+

Duck typing nedir?

+

Polimorfizmin kompozisyon ile kullanımı

+

## Hafta 7 — Tasarım Desenleri Giriş

**Bu haftada ne öğreniyorsun?** Tasarım desenleri giriş: Singleton, Factory vb.

Desen = tekrar eden çözüm şablonu

Singleton tek nesne

Factory nesne üretimi

### Basit Anlatım

- Desen = tekrar eden çözüm şablonu
- Singleton tek nesne
- Factory nesne üretimi

Genel

–

||

7. Hafta

+

7. Hafta

+

Soyutlama (Abstraction)

+

Soyutlama (Abstraction)

+

Soyutlama, bir nesnenin sadece gerekli özelliklerini sunup,

+

Soyut Sınıf Fikri

+



Soyut Sınıf Fikri

+

Polimorfizm ve soyutlama fikri mevcut.

+

Soyut Sınıf Fikri

+

Abstract Base Class (ABC) Nedir?

+

Soyut sınıflar tanımlamak için kullanılır.

+

metot varsa, o sınıf soyut

+

sınıftır.

+

Soyut sınıf = kural

+

Soyut sınıf sadece soyut metot

+

Soyut sınıf = kural + ortak altyapı

+

Soyut Sınıf

+

Soyutlama Örneği - 1

+

Soyutlama Örneği - 1

+

sınıflara bırakmak için soyutlama

+

metot olarak belirlenir

+

Soyutlama Örneği - 1

+

Soyutlama Örneği - 1

+

Soyutlama Örneği 1- Kullanım

+

Kapsülleme ve Soyutlama Birlikte Kullanım Örneği

+

Kapsülleme ve Soyutlama Birlikte Kullanım Örneği

+

Soyut Hesap Sınıfı

+

metotlar, alt sınıflar bu

+

metotları yazmak zorundadır

+

Soyutlama Kavramı

+

Soyut Sınıf ve Metot Oluşturma

+

Kapsülleme ve Soyutlama Birlikte

+

## Hafta 8 — Tasarım Desenleri Devam

**Bu haftada ne öğreniyorsun?** Observer, Decorator ve diğer desenler.

Observer olay dinler

Decorator davranış ekler

Desenler SOLID ile uyumlu olmalı

### Basit Anlatım

- Observer olay dinler
- Decorator davranış ekler
- Desenler SOLID ile uyumlu olmalı

8. Hafta

–

Öğr. Gör. Ecmel Albayrak

8. Hafta

+

`__init__` Metodu (Yapıcı Metot)

+

`__str__` Metodu (String Temsili)

+

Aritmetik Operatörler Alıştırması

+

Aritmetik Operatörler Alıştırması

+

Kalıtım (Inheritance): `isinstance()`, alt sınıfları da doğru şekilde

+

## Hafta 9 – SOLID Prensipleri

**Bu haftada ne öğreniyorsun?** PDF 9. hafta slaytları: SOLID tanımları, SRP/OCP/LSP/ISP/DIP anlatımı.

SRP tek is

OCP genişlet

LSP yerine koy

ISP küçük arayüz

DIP soyut bağımlılık

### Basit Anlatım

- SRP tek is
- OCP genişlet
- LSP yerine koy
- ISP küçük arayüz
- DIP soyut bağımlılık

9. Hafta

-

Öğr. Gör. Ecmel Albayrak

9. Hafta

+

SOLID Prensipleri

+

SOLID

+

Nesne Yönelimli Programlama (OOP) paradigmasını kullanarak

+

Tek Sorumluluk (Single Responsibility) Prensipleri

+

Tek Sorumluluk (Single Responsibility) Prensibi

+

Tek Sorumluluk (Single Responsibility) Prensibi

+

Tek Sorumluluk (Single Responsibility) Prensibi

+

Tek Sorumluluk (Single Responsibility) Prensibi

+

Single Responsibility Prensibi Alıştırma

+

Açık/Kapalı (Open/Closed) Presibi

+

Açık/Kapalı (Open/Closed) Presibi

+

Açık/Kapalı (Open/Closed) Presibi

+

sınıflar oluştururuz (açık)

+

Açık/Kapalı (Open/Closed) Presibi

+

Açık/Kapalı (Open/Closed) Presibi Alıştırma

+

adresindeki kodu open/closed prensibine uygun hale getiriniz.

+

Soyut bir Sekil sınıfı oluşturun

+

Liskov Yerine Geçme (Liskov Substitution)Prensibi

+

Liskov Yerine Geçme (Liskov Substitution) Presibi

+

sınıfın (Kanal) davranışını

+

Liskov Substitution Presibi Uygulanması

+

Liskov Substitution Presibi Kullanımı

+

Liskov Substitution Presibi Alıştırma

+

sindeki kodun liskov yerine geçme prensibine uygun hale getiriniz.

+

Arayüz Ayrımı (Interface Segregation) Prensibi

+

Arayüz Ayrımı (Interface Segregation) Presibi

+

Interface Segregation Presibi Refactor

+

Interface Segregation Presibi Kullanımı

+

Sınıf sadece ihtiyacı olan metodu içerir

+

Interface Segregation Presibi Alıştırma

+

Bağımlılıkların Tersine Çevrilmesi (Dependency Inversion) Presibi

+

Dependency Inversion Presibi

+

sınıfını değiştirmeyi gerektirir

+

Dependency Inversion Presibi

+

Dependency Inversion Kullanımı

+

soyutlamaya bağımlı

+

Dependency Inversion Presibi Alıştırma

+

prensibine uygun hale

+

SOLID prensiplerini anlama

+

## Hafta 10 – SOLID Calisma & Odev Cozumleri

**Bu haftada ne var?** Odev formatinda 5 SOLID ilkesi, PDF alistirmalari (OCP/LSP/ISP/DIP) cozumlu, LSP Solo Leveling.

Her ilke: tanim, amac, kotu/iyi kod, neden?

Sinif arkadas ornekleri: SRP, OCP, LSP, ISP, DIP (Python)

AlanHesaplayici, Dosya, Cihaz, Bildirim odevleri

Solo Leveling LSP ornegi – sinav icin

### Hafta 10 Ozeti

- Her ilke: tanim, amac, kotu/iyi kod, neden?
- Sinif arkadas ornekleri: SRP, OCP, LSP, ISP, DIP (Python)
- AlanHesaplayici, Dosya, Cihaz, Bildirim odevleri
- Solo Leveling LSP ornegi – sinav icin



Nesne Yönelimli Programlama — Sinav Odev Formatı

Ad Soyad: \_\_\_\_\_

Oğrenci No: \_\_\_\_\_

Tarih: \_\_\_\_\_

Her SOLID ilkesi: **ilke adı, tanımı, amacı, kötü/iyi kod** ve açıklama. Altında sınıf arkadaşlarının Python örnekleri (Makyaj Studyosu, Leon, Solo Leveling vb.) yer alır.

**S**

Tek Sorumluluk İlkesi

### İLKE TANIMI

Bir sınıf yalnızca tek bir sorumluluğa sahip olmalıdır; yani sınıfı değiştirmek için yalnızca tek bir nedeni olmalıdır.

### AMACI / COZMEYE CALISTIĞI PROBLEM

Buyuyen sınıflarda kodun karışmasını, bakım zorluğunu ve hata riskini azaltmak. Her modul tek bir işi yapsın; değişiklikler izole kalsın.

*Problem:* Bir sınıf hem iş mantığı hem dosya kaydı hem e-posta gönderimi yaparsa her yeni gereksinimde tüm sınıf kırılabilir.

### İLKEYE AYKIRI (KOTU) KOD

```
class Post
  def publish
    save_to_database
    send_email_to_subscribers
    write_to_log_file
  end

  def save_to_database
    # veritabanı işlemi
  end

  def send_email_to_subscribers
    # e-posta gönderimi
  end
end
```

```
end

def write_to_log_file
  # log yazma
end
end
```

**Neden kotu?** Post sinifi ayni anda veritabani, e-posta ve loglama sorumluluklarini tasiyor. E-posta servisi degisirse Post sinifi degismek zorunda kalir. Test etmek zorlasir; tek bir degisiklik birden fazla nedenden dolayi sinifi etkiler — SRP ihlali.

#### ILKEYE UYGUN (iyi) KOD

```
class Post
  def publish(publisher:, notifier:, logger:)
    publisher.save(self)
    notifier.notify_subscribers(self)
    logger.info("Post yayinlandi: #{title}")
  end
end

class PostPublisher
  def save(post)
    # sadece veritabani
  end
end

class EmailNotifier
  def notify_subscribers(post)
    # sadece e-posta
  end
end

class FileLogger
  def info(message)
    # sadece log
  end
end
```

**Neden iyi?** Her sinifin tek gorevi var: Post yayinlama kararini verir, diger isleri uzman siniflara devreder. E-posta mantigi degistiginde yalnızca EmailNotifier guncellenir. Siniflar kucuk, test edilebilir ve bakimi kolay — SRP'ye uygun.

#### SINIF ARKADAS ORNEGİ (PYTHON)

##### Makyaj Studiyosu — Tek Sorumluluk (SRP)

Tema: Makyaj salonu islemleri · Hazirlayan: Gizem Senol · Dosya:

`solid_examples/single_responsibility.py`

### KOTU KOD

```
class MakyajStudyosu:
    def __init__(self, musteri_adi, hizmet_turu, ucret):
        self.musteri_adi = musteri_adi
        self.hizmet_turu = hizmet_turu
        self.ucret = ucret

    def makyaj_uygula(self):
        print(f"{self.musteri_adi} – {self.hizmet_turu} makyaji")

    def fircalari_ve_salonu_temizle(self):
        print("Firçalar dezenfekte, salon temizleniyor")

    def fatura_kes(self):
        kdv_dahil = self.ucret * 1.20
        print(f"Fatura: {kdv_dahil} TL")
```

**Neden kötü?** Sınıfın değişmesi için 3 farklı neden var: makyaj, hijyen, finans. KDV oranı değişirse makyaj koduna dokunmak gerekir — SRP ihlali.

### İYİ KOD

```
class MakyajHizmeti:
    def makyaj_uygula(self): ...

class HijyenYoneticisi:
    def fircalari_temizle(self): ...
    def salonu_temizle(self): ...

class FinansYonetimi:
    @staticmethod
    def fatura_kes(musteri_adi, ucret):
        kdv_dahil = ucret * 1.20
        ...
```

**Neden iyi?** Her sınıf tek iş yapar: makyaj, hijyen veya fatura. Vergi kuralı değişince yalnızca FinansYonetimi güncellenir.



Acık/Kapalı İlke

### İLKE TANIMI

Yazılım varlıkları (sınıflar, modüller) genişlemeye açık, değişikliğe kapalı olmalıdır. Mevcut kodu değiştirmeden yeni davranış eklenebilmelidir.

## AMACI / COZMEYE CALISTIĞI PROBLEM

Calisan kodu bozmadan yeni ozellik eklemek. if/elsif zincirleri yerine genisleme (kalitim, modul, polymorphism) ile yeni davranislar eklenir.

*Problem:* Her yeni odeme tipi icin mevcut sinifa if/elsif eklenirse kod surekli degisir ve regression riski artar.

### ILKEYE AYKIRI (KOTU) KOD

```
class PaymentProcessor
  def pay(method, amount)
    if method == :credit_card
      # kredi karti odeme
    elsif method == :paypal
      # paypal odeme
    elsif method == :bank_transfer
      # havale – her yeni tip icin buraya ekleme yapilir
    end
  end
end
```

**Neden kotu?** Yeni bir odeme yontemi (ornegin :crypto) eklendiginde PaymentProcessor sinifinin icindeki pay metodu degistirilmek zorunda. Acik/kapali ilkeye gore sinif genislelemeye kapali degil, her ekleme mevcut kodu kirar — OCP ihlali.

### ILKEYE UYGUN (iyi) KOD

```
class PaymentProcessor
  def pay(gateway, amount)
    gateway.charge(amount)
  end
end

class CreditCardGateway
  def charge(amount)
    # kredi karti
  end
end

class PaypalGateway
  def charge(amount)
    # paypal
  end
end

# Yeni tip: mevcut kodu degistirmeden ekle
class CryptoGateway
  def charge(amount)
    # kripto odeme
  end
end
```

```
end
end
```

**Neden iyi?** PaymentProcessor artık hangi ödeme tipi olduğunu bilmez; charge metoduna sahip her gateway çalışır. CryptoGateway eklemek için PaymentProcessor'a dokunulmaz. Mevcut kod kapalı (degismez), yeni davranış genişlemeyle açık — OCP'ye uygun.

## SINIF ARKADAS ORNEGİ (PYTHON)

### Para Transferi — Açık/Kapalı (OCP)

Tema: Banka transfer yöntemleri · Hazırlayan: Salih KOZA · Dosya:

`solid_examples/open_closed.py`

#### KOTU KOD

```
class ParaTransferi:
    def transfer_yap(self, miktar, yontem):
        if yontem == "havale":
            print(f"{miktar} TL Havale. Komisyon: 0")
        elif yontem == "eft":
            print(f"{miktar} TL EFT. Komisyon: 5")
        else:
            raise ValueError("Bilinmeyen yontem!")
```

**Neden kötü?** SWIFT veya kripto eklemek için sınıfı açıp yeni elif yazmak gerekir. Çalışan havale/EFT kodu risk altında — OCP ihlali, if/elif spaghetti.

#### İYİ KOD

```
class TransferYontemi(ABC):
    @abstractmethod
    def islem_yap(self, miktar): pass

class Havale(TransferYontemi):
    def islem_yap(self, miktar): ...

class EFT(TransferYontemi):
    def islem_yap(self, miktar): ...

class TransferMerkezi:
    def transferi_baslat(self, miktar, yontem: TransferYontemi):
        yontem.islem_yap(miktar) # if/elif yok!
```

**Neden iyi?** TransferMerkezi degismez; yeni yontem için sadece yeni sınıf eklenir (örneğin class SwiftTransfer). Mevcut kod kapalı, genişleme açık — OCP uygun.



## İLKE TANIMI

Alt sınıf nesneleri, üst sınıf nesnelerinin yerine kullanılabilir; yerine konulduğunda programın davranışı bozulmamalı.

## AMACI / COZMEYE CALISTIĞI PROBLEM

Kalitim hiyerarsisinde alt sınıfın üst sınıfın sözleşmesini (contract) bozmamasını sağlamak. Polimorfizm güvenli çalışsın. Sung Jinwoo da bir Avci — yerine konunca program çalışıyor mu?

*Problem:* YanlisPlayer güc artırıncı zeka da artırıyor; Avci bekleyen stat\_test\_et patlar. Alt sınıf üst sınıfın yerine konulduğunda beklenmeyen davranış → LSP ihlali.

### İLKEYE AYKIRI (KOTU) KOD

```
class Avci_Kotu:
  def guc_artir(miktar)
    @guc += miktar
  end
  def zeka_artir(miktar)
    @zeka += miktar
  end
end

class YanlisPlayer < Avci_Kotu
  def guc_artir(miktar)
    @guc += miktar
    @zeka += miktar # BEKLENMEDİK!
  end
end

# stat_test_et: guc+10, zeka+5 → guc=20, zeka=15 beklenir
# YanlisPlayer → zeka=20 → KALDI
```

**Neden kötü?** Avci'da güc ve zeka bağımsız artar. YanlisPlayer güc artırıncı zeka da artırır. Üst sınıfı bekleyen test/stat fonksiyonu patlar. Alt sınıf yerine konulamaz — LSP ihlali. Detaylı Solo Leveling örneği: site [#lsp-solo-leveling](#) bölümü.

### İLKEYE UYGUN (İYİ) KOD

```
class Hunter
  def dungeon_temizle(seviye)
    raise NotImplementedError
  end
  def skill_kullan
    raise NotImplementedError
  end
end
```

```

    end
end

class ERankHunter < Hunter
  def dungeon_temizle(seviye)
    seviye <= 2
  end
  def skill_kullan
    "temel saldırı"
  end
end

class SungJinwoo < Hunter
  def dungeon_temizle(seviye)
    @golge_sayisi += 1
    true # sozlesme: bool doner
  end
  def skill_kullan
    "Arise kullandı"
  end
end

# baskani_hazirla(hunter) – hangi hunter gelirse gelsin calisir

```

**Neden iyi?** Her Hunter ayni sozlesmeye uyar: dungeon\_temizle bool, skill\_kullan string doner. SungJinwoo ekstra ozellik ekler ama sozlesmeyi bozmaz. Alt sinif ust sinifin yerine guvenle konulabilir — LSP'ye uygun.

## SİNİF ARKADAS ORNEGİ (PYTHON)

### Solo Leveling Avci — Liskov (LSP)

Tema: Solo Leveling / Hunter hiyerarsisi · Hazırlayan: Muhammed Hamza Erhan · Dosya:

[solid\\_examples/liskov.py](#)

#### KOTU KOD

```

class Avci_Kotu:
    def guc_artir(self, miktar):
        self.guc += miktar
    def zeka_artir(self, miktar):
        self.zeka += miktar

class YanlisPlayer(Avci_Kotu):
    def guc_artir(self, miktar):
        self.guc += miktar
        self.zeka += miktar # beklenmeyen!

def stat_test_et(avci):
    avci.guc_artir(10); avci.zeka_artir(5)

```

```
# Beklenen: guc=20, zeka=15
# YanlisPlayer -> zeka=20 -> KALDI
```

**Neden kotu?** Avci'da guc ve zeka bagimsiz artar. YanlisPlayer guc artirince zeka da artar; stat\_test\_et patlar. Alt sinif ust sinifin yerine konulamaz — LSP ihlali.

#### İYİ KOD

```
class Hunter(ABC):
    @abstractmethod
    def dungeon_temizle(self, seviye: int) -> bool: pass
    @abstractmethod
    def skill_kullan(self) -> str: pass

class ERankHunter(Hunter): ...
class SungJinwoo(Hunter): ...

def baskani_hazirla(hunter: Hunter, seviye: int):
    if hunter.dungeon_temizle(seviye):
        print(hunter.skill_kullan())
# Hangi hunter gelirse gelsin calisir
```

**Neden iyi?** Her Hunter bool/str sozlesmesine uyar. SungJinwoo ekstra ozellik ekler ama sozlesmeyi bozmaz. Detay: asagidaki LSP Solo Leveling paneli.

[Detayli LSP ornegi ve 5 kural ->](#)



Arayuz Ayirimi İlkesi

#### İLKE TANIMI

Istemiciler kullanmadiklari arayuzlere bagimli olmamali. Buyuk, siskin arayuzler yerine kucuk, ozellestirilmis arayuzler tercih edilmeli.

#### AMACI / COZMEYE CALISTIĞI PROBLEM

Siniflari zorunlu ama bos/kullanilmayan metot implementasyonlarından kurtarmak. Modul ve mixin'lerde sadece gereken davranislar sunulur.

*Problem:* Tek dev modul hem yazdir hem taray hem faks metotlari icerirse bazı siniflar bos metot birakmak zorunda kalir.

#### İLKEYE AYKIRI (KOTU) KOD

```
module Worker
  def work; end
```



```

    def eat; end
    def sleep; end
    def attend_meeting; end # her worker icin gerekli degil
end

class Robot
  include Worker

  def work
    "Calisiyor"
  end

  def eat
    # robot yemek yemez – bos veya anlamsiz
  end

  def sleep
    # robot uyumaz
  end

  def attend_meeting
    # anlamsiz
  end
end

```

**Neden kotu?** Robot, kullanmadigi eat/sleep/attend\_meeting metotlarini da implement etmek zorunda. Buyuk arayuz istemciyi gereksiz bagimliliklere zorlar. ISP: kullanilmayan metotlar arayaze dahil edilmemeli.

#### ILKEYE UYGUN (iyi) KOD

```

module Workable
  def work; end
end

module Eatable
  def eat; end
end

class Human
  include Workable
  include Eatable

  def work; "Calisiyor"; end
  def eat; "Yemek yiyor"; end
end

class Robot
  include Workable

```

```
def work; "Calisiyor"; end
# sadece ihtiyaci olan modul
end
```

**Neden iyi?** Arayuzler role gore ayrildi: Workable, Eatable. Robot yalnızca Workable alır; bos eat/sleep yazmak zorunda kalmaz. Her sinif sadece ihtiyaci olan davranisa bagimli — ISP'ye uygun.

## SİNİF ARKADAS ORNEGİ (PYTHON)

### Hayvan Arayuzleri — Arayuz Ayirimi (ISP)

Tema: Kartal, kopek, kedi davranislari · Hazirlayan: Hilmi Arda Dagci · Dosya:

[solid\\_examples/Interface\\_segration.py](#)

#### KOTU KOD

```
class Animal(ABC):
    @abstractmethod
    def eat(self): pass
    @abstractmethod
    def fly(self): pass
    @abstractmethod
    def swim(self): pass

class Eagle(Animal):
    def eat(self): ...
    def fly(self): ...
    def swim(self): pass # Kartal yuzmez!

class Dog(Animal):
    def eat(self): ...
    def fly(self): pass # Kopek ucamaz!
    def swim(self): ...
```

**Neden kotu?** Eagle, Dog ve Cat kullanmadigi fly/swim metotlarini bos implement etmek zorunda. Siskin arayuz gereksiz kod ve hata riski — ISP ihlali.

#### İYİ KOD

```
class Eatable(ABC):
    @abstractmethod
    def eat(self): pass

class Flyable(ABC):
    @abstractmethod
    def fly(self): pass

class Swimmable(ABC):
```

```
@abstractmethod
def swim(self): pass
```

```
class Eagle(Eatable, Flyable): ...
class Dog(Eatable, Swimmable): ...
class Cat(Eatable): ...
```

**Neden iyi?** Her sınıf yalnızca ihtiyacı olan küçük arayuzu alır. Kedi yüzme/uçma metodu yazmak zorunda kalmaz — ISP uygun.

D

Bagimlilik Tersine Cevirme Ilkesi

### ILKE TANIMI

Ust seviye moduller alt seviye modullere bagimli olmamali; her ikisi de soyutlamalara (abstraction) bagimli olmalidir.

### AMACI / COZMEYE CALISTIGI PROBLEM

Siki bagimliliklari (concrete class) gevsetmek; test, degistirme ve genisletmeyi kolaylastirmak. Dependency Injection ile somut sinif yerine arayuz/soyut bagimlilik kullanilir.

*Problem:* Controller dogrudan PostgreSQL sinifina bagliysa MySQL'e gecis tum controller'i degistirir.

### ILKEYE AYKIRI (KOTU) KOD

```
class PostsController
  def create
    db = PostgreSQLConnection.new # somut sinifa dogrudan bagimli
    db.insert(params[:post])
  end
end
```

**Neden kotu?** PostsController dogrudan PostgreSQLConnection somut sinifina bagimli. Veritabani degirse controller degismek zorunda. Ust seviye (controller) alt seviyeye (PostgreSQL) bagimli — DIP ihlali.

### ILKEYE UYGUN (iyi) KOD

```
class PostsController
  def initialize(repository:)
    @repository = repository # soyut bagimlilik enjekte edilir
  end

  def create(params)
```

```

    @repository.insert(params)
  end
end

class PostRepository
  def insert(data)
    # PostgreSQL veya baska implementasyon
  end
end

# Rails'de benzeri:
# @post = Post.new(post_params) → ActiveRecord soyutlamasi
PostsController.new(repository: PostRepository.new)

```

**Neden iyi?** Controller somut veritabani sinifini bilmez; repository arayuzune bagimlidir. Testte sahte (mock) repository enjekte edilebilir. Ust ve alt seviye soyutlamaya bagimli — DIP'ye uygun. Rails'de ActiveRecord da bu soyutlamadir.

## SINIF ARKADAS ORNEGİ (PYTHON)

### Leon & Silah — Bagimlilik Tersine Cevirme (DIP)

Tema: Resident Evil — Leon silah secimi · Hazirlayan: Emir Can BICEN · Dosya:

[solid\\_examples/Dependency\\_inversion.py](#)

#### KOTU KOD

```

class Tabanca:
    def ates_et(self):
        print("BANG!")

class Leon:
    def __init__(self):
        self.silah = Tabanca() # somut sinifa bagimli

    def saldir(self):
        self.silah.ates_et()

```

**Neden kotu?** Leon yalnızca Tabanca kullanabilir. Shotgun veya Requiem eklemek icin Leon sinifini degistirmek gerekir — ust seviye alt seviyeye bagimli, DIP ihlali.

#### IYİ KOD

```

class Silah(ABC):
    @abstractmethod
    def ates_et(self): pass

class Tabanca(Silah): ...
class Shotgun(Silah): ...

```

```
class Requiem(Silah): ...
```

```
class Leon:  
    def __init__(self, silah: Silah):  
        self.silah = silah # disaridan enjekte
```

```
leon_boss = Leon(Requiem())  
leon_boss.saldir()
```

**Neden iyi?** Leon somut silahi bilmez; Silah soyutlamasına bagimlidir. Yeni silah = yeni sinif, Leon degismez — Dependency Injection ile DIP uygun.

## 2. PDF Alistirmalari — OCP, LSP, ISP, DIP (Cozumlu)

+

## 3. LSP Solo Leveling Detay Ornegi

+